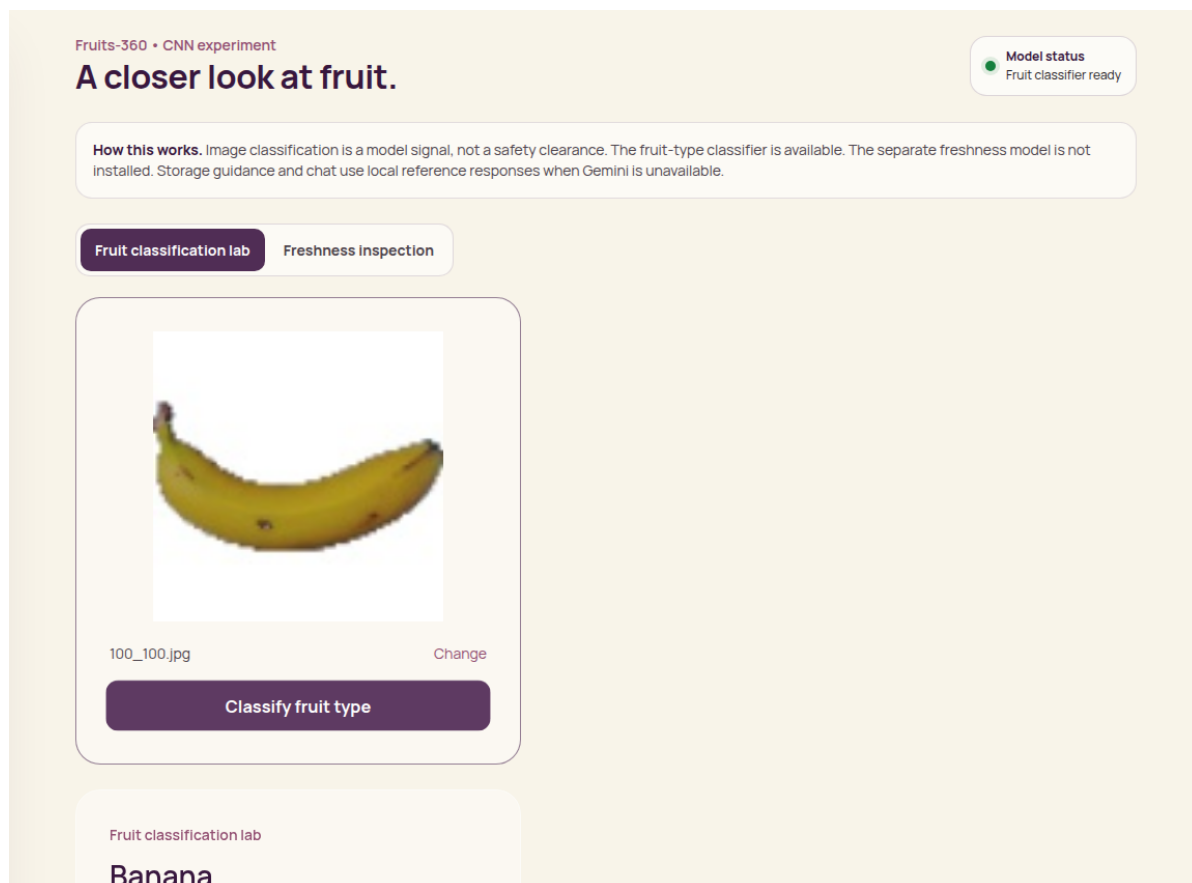


ProotyPie Fruit classification, examined.

Machine-learning mini-project • Illustrated submission report

A controlled comparison of activation functions and SGD learning rates, demonstrated through a working fruit-classification application.



Working application capture: an uploaded Fruits-360 banana classified by the saved CNN. No prediction shown here is mocked.

Four fruit classes. Four activation functions. Three learning rates. Twelve measured experiments.

Scope: a documented Fruits-360 subset. This study classifies fruit type; it does not determine freshness or food safety.

Project at a glance

Abstract. This project investigates how activation functions and learning rates affect a compact convolutional neural network. Twelve configurations were trained on the same four-class Fruits-360 subset. Configuration selection used validation macro F1; the selected configuration was refitted on training plus validation images before official-Test subset evaluation. A Next.js interface calls a Python API that loads the trained Keras model.

Assignment requirement	Evidence in this submission
CNN on a public dataset	Two convolutional layers; four Fruits-360 classes
Four activation functions	ReLU, Sigmoid, Tanh and Leaky ReLU
Gradient descent and learning rates	Mini-batch SGD at 0.001, 0.01 and 0.1
Accuracy, precision, recall and F1	All 12 validation configurations; selected model test metrics
Loss, accuracy and confusion plots	24 learning curves, comparison chart and test confusion matrix
Application demonstration	Real upload-to-result screenshots and JSON export verification

Reading guide: dataset and controls; architecture and optimization; comparison results; learning curves; test evaluation; application screenshots; verification and limitations; reproduction and references.

Dataset & experimental controls

Source: Fruits-360, Horea94/Fruit-Images-Dataset on GitHub. The experiment uses Apple Braeburn, Banana, Orange and Strawberry. Images are converted to RGB, resized to 48×48 pixels and scaled to $[0, 1]$.

Split	Per class	Total	Role
Training	80	320	Gradient updates
Validation	20	80	Configuration selection
Official Test subset	40	160	Final evaluation

Sampling is deterministic: the first 100 lexicographically sorted Training JPGs and first 40 Test JPGs per class. Training images are stratified into an 80/20 train/validation split with seed 20260919. SHA-256 checks reject identical files across splits.

Controls: identical architecture, initialization seed, batch size 16 and six epochs for every comparison. Only the hidden activation and learning rate vary. The output always uses softmax. The selected configuration is refitted on all 400 development images.

Important qualification: exact-file hashing does not establish independence between visually similar rotational views. This is a narrow, clean-background experiment, not the complete Fruits-360 benchmark. Random seeds were fixed, but repeated-seed uncertainty was not measured.

Architecture & optimization

Stage	Operation / output
Input	48 × 48 × 3 RGB image
Feature extraction 1	Conv2D: 8 filters, 3 × 3, same padding; chosen activation
Downsampling	MaxPooling2D: reduces spatial dimensions
Feature extraction 2	Conv2D: 16 filters, 3 × 3; same chosen activation
Aggregation	GlobalAveragePooling2D: one value per feature map
Decision	Dense: four softmax probabilities

Training objective: sparse categorical cross-entropy. Mini-batch SGD updates each parameter in the direction opposite the loss gradient: $\text{new weight} = \text{old weight} - \text{learning rate} \times \text{gradient}$. A larger learning rate takes bigger steps, but can also make optimization unstable.

ReLU keeps positive values and zeros negative values. Sigmoid compresses values into (0, 1). Tanh maps values into (−1, 1). Leaky ReLU retains a small negative slope (0.1). These choices affect the signals and gradients flowing through the convolutional layers.

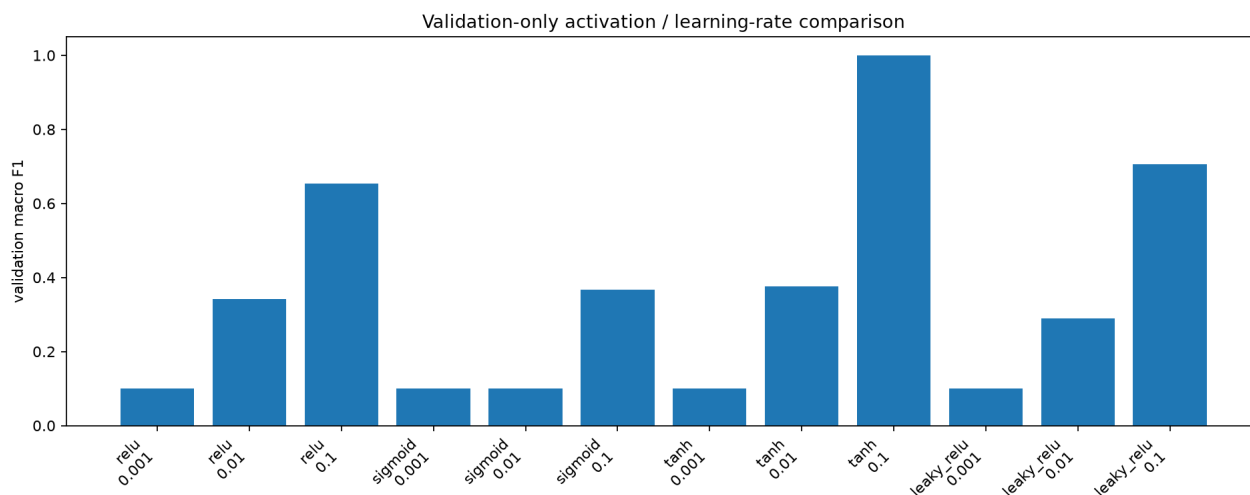
Application flow: browser upload → same-origin Next.js API proxy → Python image validation → saved Keras CNN → class probabilities → result card / JSON download. Gemini is not required for this classification path.

Twelve configurations. One comparison.

Activation	LR	Accuracy	Precision*	Recall*	F1*
relu	0.001	0.2500	0.0625	0.2500	0.1000
relu	0.01	0.4500	0.3281	0.4500	0.3413
relu	0.1	0.7500	0.5929	0.7500	0.6540
sigmoid	0.001	0.2500	0.0625	0.2500	0.1000
sigmoid	0.01	0.2500	0.0625	0.2500	0.1000
sigmoid	0.1	0.4875	0.3320	0.4875	0.3670
tanh	0.001	0.2500	0.0625	0.2500	0.1000
tanh	0.01	0.5000	0.3347	0.5000	0.3766
tanh	0.1	1.0000	1.0000	1.0000	1.0000
leaky_relu	0.001	0.2500	0.0625	0.2500	0.1000
leaky_relu	0.01	0.3875	0.3225	0.3875	0.2898
leaky_relu	0.1	0.7750	0.8515	0.7750	0.7064

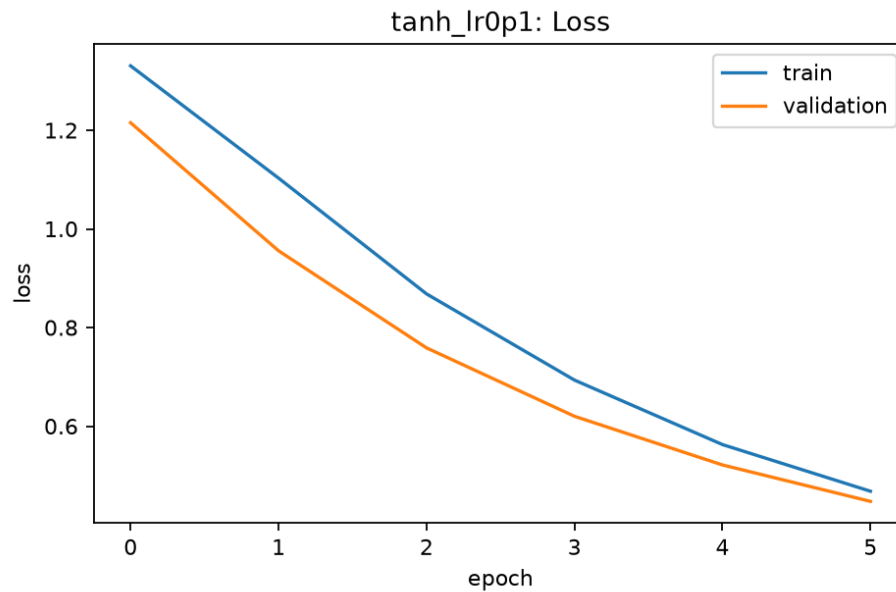
* Macro averaging gives every fruit class equal weight. These are validation metrics, not twelve separate test-set evaluations.

Tanh with SGD learning rate 0.1 had the highest validation macro F1 (1.0000). The lowest learning rate left all four activations at 25% validation accuracy after six epochs. That is evidence of inadequate learning within this budget—not proof that those configurations could never converge.

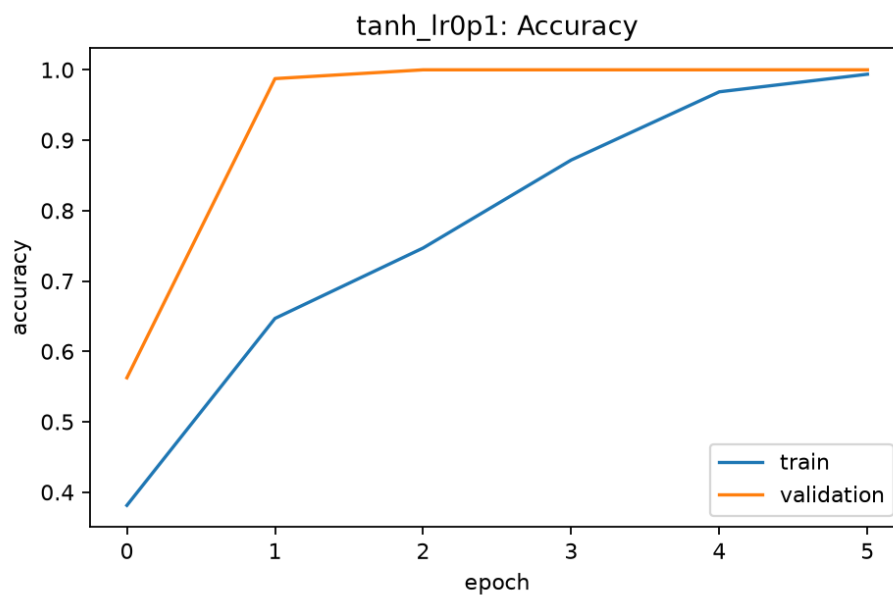


Validation-only model comparison. Higher is better; all runs share the same data split and epoch budget.

Learning behaviour



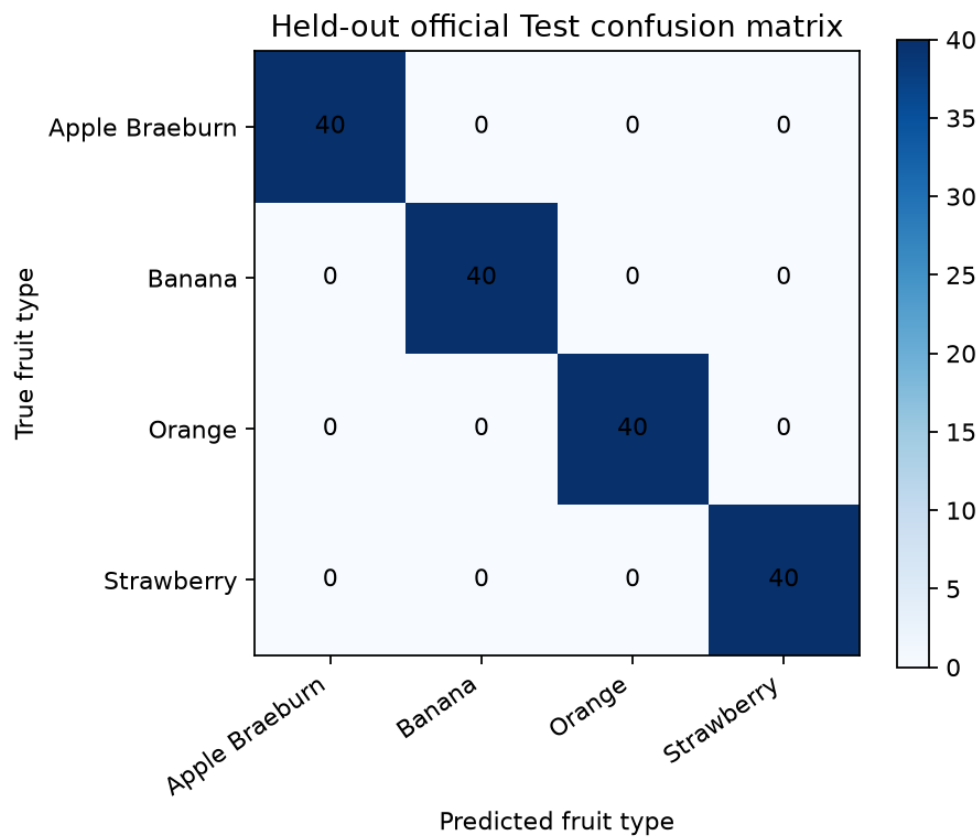
Selected configuration: training and validation cross-entropy over six epochs.



Selected configuration: training and validation accuracy. Epoch indices on these saved plots begin at zero.

The selected run ends with training loss 0.4681 and validation loss 0.4476. Higher learning rates performed better within the tested short budget. Six epochs are not enough to establish asymptotic convergence; no universal best activation is claimed.

Held-out evaluation



Confusion matrix for the final refitted model on 160 official-Test subset images: 40 images in each class.

Metric	Measured value
accuracy	1.0000
precision macro	1.0000
recall macro	1.0000
f1 macro	1.0000

All 160 selected test images were correctly classified. This is a real result on a small, visually constrained subset—not a promise of perfect performance on unseen fruit varieties, backgrounds or phone-camera images. The model is closed-set and will assign even unknown objects to one of its four classes.

The working application

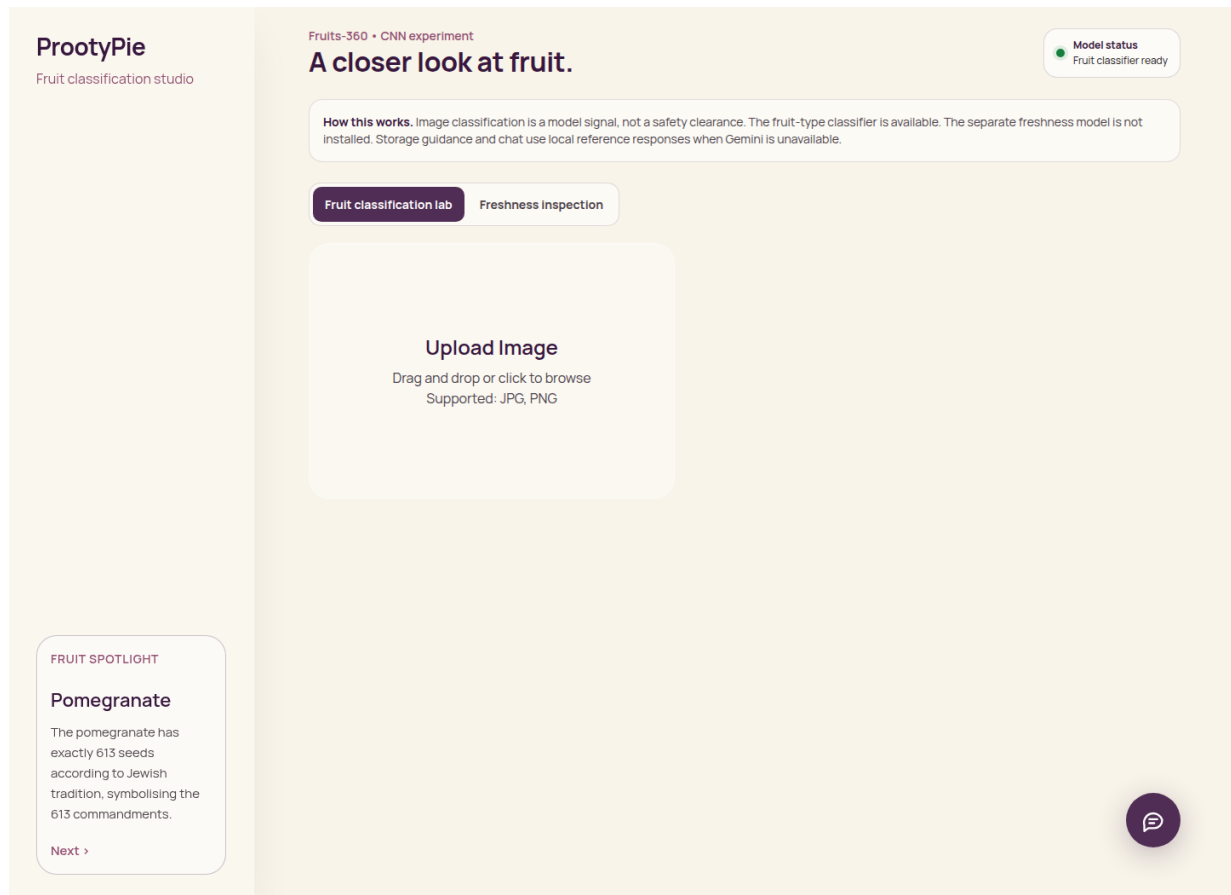
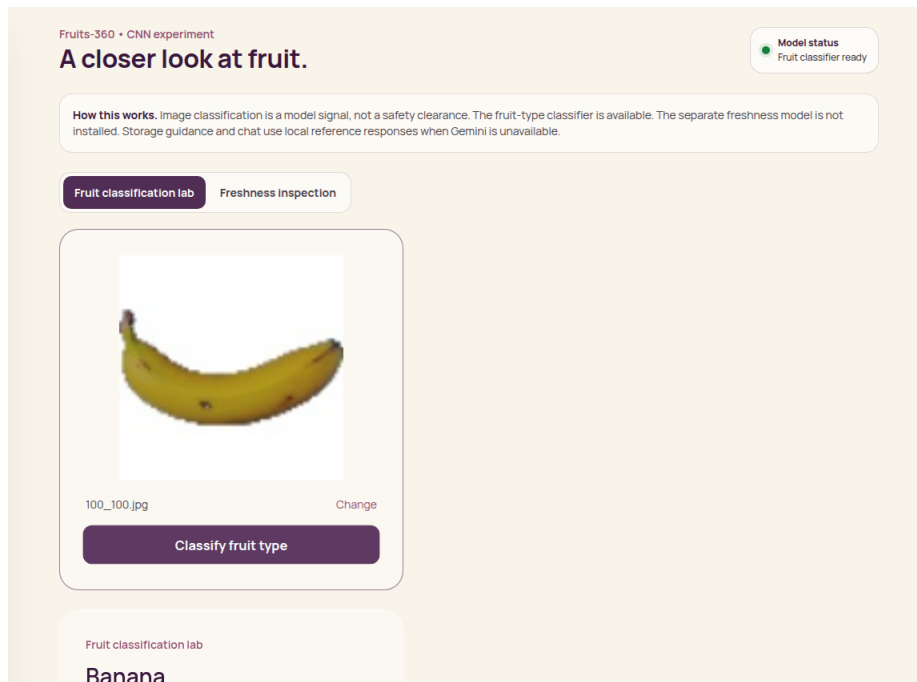


Figure: production-build dashboard, with working fruit-classifier readiness status.

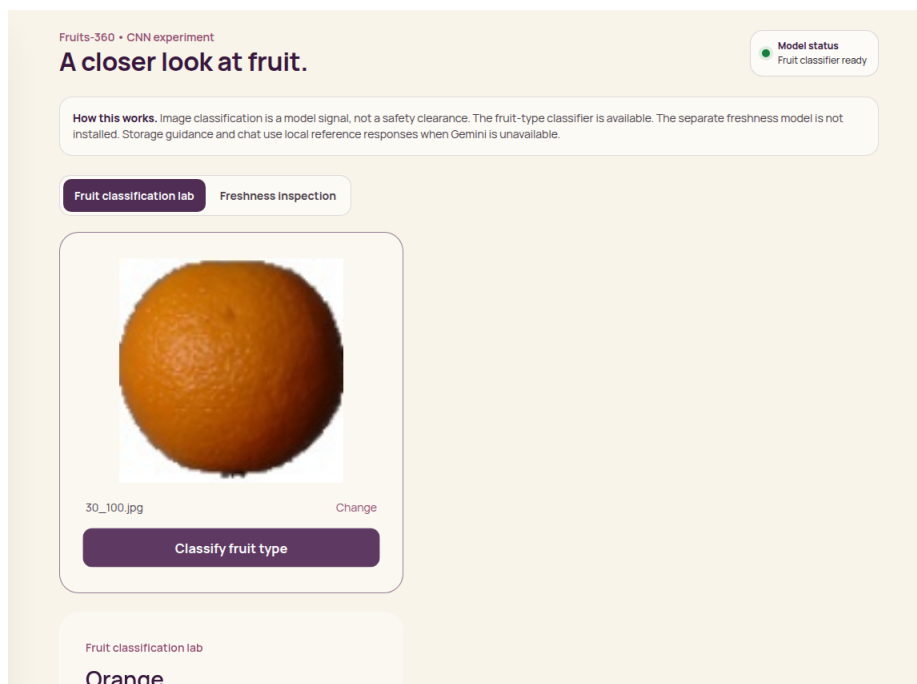
The interface keeps image upload, preview, classification, model status and result export together. The fruit-classification lab is selected by default. The legacy freshness path is separate and explicitly unavailable without its original model and configuration.

A port mismatch in the API proxy was corrected: the frontend now forwards to the actual default backend at 127.0.0.1:8000. Screenshots were captured from the running production build, not from a design mockup.

Results in the interface



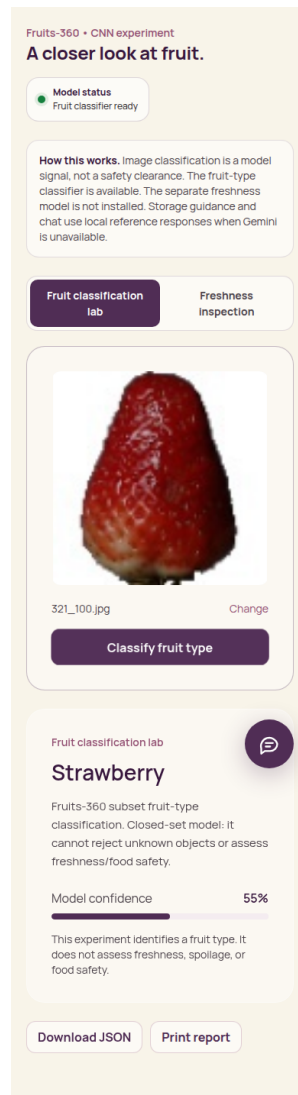
Banana: real upload 100_100.jpg; predicted Banana; model confidence 93.12%.



Orange: real upload 30_100.jpg; predicted Orange; model confidence 82.48%.

Confidence is the model output for one image. It is neither aggregate test accuracy nor a food-safety score.

Mobile & export verification



Mobile result at a 390-pixel viewport. No horizontal overflow was detected.

The Strawberry upload returned a real classification response. The Download JSON button was exercised and its file saved as reports/screenshots/export-example.json. Browser evidence and endpoint responses are retained in reports/screenshots/qa.json.

Verification & limitations

Check	Outcome
Backend and experiment unit tests	13 passed in the previous combined run; rechecked for this revision
Frontend production build	Passed after proxy / readiness changes
Browser upload-to-result	Banana, Orange and Strawberry returned valid CNN results
JSON export	Downloaded successfully through the real button
Mobile layout	390-pixel viewport; no horizontal overflow

The original fresh/rotten model is missing. This report makes no freshness, shelf-life or food-safety performance claim. Gemini-backed features were not tested and are not required for the demonstrated classification flow.

Study limitations: four classes only; deterministic rather than representative sampling; one random seed; short training budget; possible similarity among rotational images; no external-camera test set; no unknown-object rejection; no significance test.

Conclusion. The comparison demonstrates the required CNN workflow and measurable effects of activation and learning-rate choice. Tanh with SGD at 0.1 was best under this particular setup. Extending classes, training duration, seeds and external-image evaluation would strengthen generalization evidence, but is not claimed as completed here.

Reproduce & present

Use Python 3.11 or 3.12 (the legacy API uses cgi, removed in Python 3.13). Install dependencies from requirements.txt. Start the backend with `python api/server.py`. In frontend, run `npm ci`, `npm run build` and `npm start`. The frontend default proxy targets port 8000.

To rerun all experiments: `python experiments/run_fruits360_experiment.py --run`. The notebook `notebooks/Fruits360_CNN_Experiments.ipynb` invokes the same training pipeline. Results, model and class order are saved under `artifacts/experiment/`.

Suggested demonstration: open the classification lab; upload a held-out Banana image; run classification; explain the confidence-versus-accuracy distinction; download JSON; show the validation comparison and confusion matrix; close with the subset and freshness limitations.

Primary sources and evidence

Dataset: <https://github.com/Horea94/Fruit-Images-Dataset>

Source splits: /Training and /Test in that repository

Training implementation: `experiments/run_fruits360_experiment.py`

Measured results: `artifacts/experiment/summary.json`

Browser captures: `reports/screenshots/`

Project: <https://github.com/Adhi1106/ProotyPie---Fruit-Quality-Inspection>

Original repository author: Adhira Praveen, as credited in the original README. No student register number, institution, supervisor signature or experimental outcome has been invented.